Algorithmica (2009) 55: 395–421
DOI 10.1007/s00453-008-9168-0

A Polynomial Quantum Algorithm for Approximating the Jones Polynomial

Dorit Aharonov · Vaughan Jones · Zeph Landau

Received: 15 April 2007 / Accepted: 10 January 2008 / Published online: 19 March 2008
© Springer Science+Business Media, LLC 2008

Abstract The Jones polynomial, discovered in 1984, is an important knot invariant in topology. Among its many connections to various mathematical and physical areas, it is known (due to Witten) to be intimately connected to Topological Quantum Field Theory (TQFT). The works of Freedman, Kitaev, Larsen and Wang provide an efficient simulation of TQFT by a quantum computer, and vice versa. These results implicitly imply the existence of an efficient (namely, polynomial) quantum algorithm that provides a certain additive approximation of the Jones polynomial at the fifth root of unity, $e^{2\pi i/5}$, and moreover, that this problem is BQP-complete. Unfortunately, this important algorithm was never explicitly formulated. Moreover, the results of Freedman et al. are heavily based on TQFT, which makes the algorithm essentially inaccessible to computer scientists.

We provide an explicit and simple polynomial quantum algorithm to approximate the Jones polynomial of an n strands braid with m crossings at any primitive root of unity $e^{2\pi i/k}$, where the running time of the algorithm is polynomial in m, n and k. Our algorithm is based, rather than on TQFT, on well known mathematical results (specifically, the path model representation of the braid group and the uniqueness of the Markov trace for the Temperley-Lieb algebra). By the results of Freedman et al., our algorithm solves a BQP complete problem.

Our algorithm works by encoding the local structure of the problem into the local unitary gates which are applied by the circuit. This structure is significantly different from previous quantum algorithms, which are mostly based on the Quantum Fourier

D. Aharonov (✉)
School of Computer Science and Engineering, The Hebrew University, Jerusalem, Israel
e-mail: doria@cs.huji.ac.il

V. Jones
Department of Mathematics, U.C. Berkeley, Berkeley, CA, USA

Z. Landau
Department of Mathematics, The City College of New York, New York, NY, USA

transform. Since the results of the current paper were presented in their preliminary form, these ideas have been extended and generalized in several interesting directions. Most notably, Aharonov, Arad, Eban and Landau give a simplification and extension of these results that provides additive approximations for all points of the Tutte polynomial, including the Jones polynomial at any point, and the Potts model partition function at any temperature and any set of coupling strengths. We hope and believe that the ideas presented in this work will have other extensions and generalizations.

1 Introduction

Since Shor's breakthrough discovery in 1994 [35], quantum algorithms with an exponential speedup over the best known classical algorithms have been shown for a number of problems (e.g., [9, 19, 29, 38]). All these problems and algorithms share some common features: the problems are group or number theoretic in nature and the key component of each algorithm is the quantum Fourier transform.[1] Arguably, the greatest challenge of quantum computation is the discovery of new algorithmic techniques.

In this paper we describe a polynomial time quantum algorithm that approximates the #P-hard problem of evaluating the Jones polynomial at certain roots of unity. The best known classical algorithm for this approximation problem is exponential. Our algorithm is significantly different from all previously known quantum algorithms that achieve an exponential speed up in the following three important ways: (1) it solves a problem which is combinatorial rather than group or number theoretic in nature, (2) it does so not by using the Fourier transform, but instead, by exploiting a certain structure of the problem and encoding it into the nature of the unitary gates being used, and (3) it solves a problem that is BQP-hard [11], that is, a problem that captures all the power of the quantum model.

The connection between quantum computation and the Jones polynomial was first made through the series of papers [10–13]. A model of quantum computation based on Topological Quantum Field Theory (TQFT) and Chern-Simons theory was defined in [10, 11], and Kitaev, Larsen, Freedman and Wang showed that this model is polynomially equivalent in computational power to the standard quantum computation model in [12, 13]. These results, combined with a deep connection between TQFT and the value of the Jones polynomial at particular roots of unity discovered by Witten 20 years ago [39], implicitly implies an efficient[2] quantum algorithm for the approximation of the Jones polynomial at the value $e^{2\pi i/5}$. This connection is also discussed, from the point of view of TQFT, in Preskill's notes [33]. Unfortunately, the important quantum algorithm implied by these intriguing results, though referred to in [6], was never explicitly formulated.

In this paper we use a different route to connect quantum computation and the Jones polynomial, one that does not involve TQFT. We present an explicit and simple

[1] One interesting exception is the beautiful result of [7] who used random walks to achieve an exponential algorithmic speed-up for an oracle graph problem. This technique, however, has not yet found applications.

[2] See Sect. 2.1 for a precise explanation of this term.

to state algorithm for the above problem, which is based purely on algebraic results from more than 20 years ago. Moreover, our algorithm works for all roots of unity of the form $e^{2\pi i/k}$, for all polynomial k's, going beyond the discussions in previous works involving only constant k's. If one is familiar with TQFT, one can easily detect the strong connection between the two algorithms, but we do not explain this here. We now describe the precise problem that we solve.

1.1 Background on the Jones Polynomial

A central issue in low dimensional topology is that of *knot invariants*. A knot invariant is a function on knots (or links—i.e. circles embedded in R^3) which is invariant under isotopy of the link, i.e., it does not change under stretching, moving, etc., but no cutting. In 1984, Jones [22] discovered a new knot invariant, now called the Jones polynomial $V_L(t)$, which is a Laurent polynomial in $\sqrt{t}$ with integer coefficients, and which is an invariant of the link L. In addition to the important role it has played in low dimensional topology, the Jones polynomial has found applications in numerous fields, from DNA recombination [32], to statistical physics [41].

The Jones polynomial can also be defined as a function of braids. A braid of n strands and m crossings is described pictorially by n strands hanging alongside each other, with m crossings, each of two adjacent strands. A braid may be "closed" to form a link by tying its ends together. In this paper we will be interested in two ways to perform such closures, namely, the *trace closure* and the *plat closure* (to be defined later). We will be interested in the Jones polynomial of links that are trace or plat closures of braids.

The essential component of the value of the Jones Polynomial of a link L at the point t can be computed by the following process: project L to the plane keeping track of crossings to get what is called a *link diagram*. Now replace every crossing  by a particular linear combination (with coefficients being functions of the parameter t) of the pictures  and  . The result is a linear combination of diagrams containing only closed loops. Replace each of these diagrams with a particular function of the parameter t and the number of loops. The resulting function of t is a scaled version of the Jones polynomial $V_L(t)$.

From the moment of the discovery of the Jones polynomial, the question of how hard it is to compute was important. When the number of strands, n, is bounded, there is an easy efficient classical algorithm to compute the Jones polynomial exactly, but we are interested in the general n case. There is a very simple inductive algorithm (essentially due to Conway [8]) to compute the Jones polynomial by changing crossings in a link diagram, but, naively applied, this takes exponential time in the number of crossings. It was shown [20] that the computation of $V_L(t)$ is #P-hard for all but a few values of t where $V_L(t)$ has an elementary interpretation. Thus a polynomial time algorithm for computing $V_L(t)$ for any value of t other than those elementary ones is unlikely. Of course, the #P-hardness of the problem does not rule out the possibility of good approximations; see, e.g., [21]. Still, the best classical algorithms to approximate the Jones polynomial at all but trivial values are exponential.

1.2 Our Results

We show an efficient, explicit, and simple quantum algorithm to approximate the Jones polynomial at all points of the form $t = e^{2\pi i/k}$. It will in fact be easier to use the parameter $A = t^{-1/4} = ie^{-\pi i/2k}$. We prove the following for the trace closure B^{tr} and the plat closure B^{pl} of a braid B:

Theorem 1.1 *There exists a quantum algorithm which, for a given braid B with n strands and m crossings, and a given integer k, is polynomial in m, n and k, and with all but exponentially small probability, outputs a complex number r with $|r - V_{B^{\mathrm{tr}}}(e^{2\pi i/k})| < \varepsilon d^{n-1}$ where $d = -A^2 - A^{-2}$, and ε is inverse polynomial in n, k, m.*

Theorem 1.2 *There exists a quantum algorithm which, for a given braid B with n strands and m crossings, and a given integer k, is polynomial in m, n and k, and with all but exponentially small probability, outputs a complex number r with $|r - V_{B^{pl}}(e^{2\pi i/k})| < \varepsilon d^{n/2-1}$ where $d = -A^2 - A^{-2}$ and ε is inverse polynomial in n, k, m.*

We remark that the approximation we provide here is additive, namely the result lies in a given window, whose size is independent of the actual value we are trying to approximate. This of course is much weaker than a multiplicative approximation, which is what one might desire (see discussion in [6]). One might wonder if under such weak requirements, the problem remains meaningful at all. It turns out that, in fact, this additive approximation problem is BQP-hard (see Sect. 2.2 for an explanation of this term, and related terms such as BQP, and BQP-complete).

Theorem 1.3 Adapted from Freedman, Larsen and Wang [13] *The problem of approximating the Jones polynomial of the plat closure of a braid at $e^{2\pi i/k}$ for constant k, to within the accuracy given in Theorem 1.2, is BQP-hard.*

This result was recently strengthened by Aharonov and Arad [1] to any k which is polynomial in the size of the input. This means that BQP-hardness holds for all the plat closure cases for which our algorithm is efficient. The implication is that the problem discussed in Theorem 1.2 is BQP-complete, namely, it exactly captures the computational power of polynomial time quantum computation. This is one of only a few examples known today, of non-trivial BQP-complete problems.

Curiously, the hardness results of [1, 13] are not known to hold (regardless of k) for the approximation of the trace closure. In other words, the problem which is studied in Theorem 1.1 is not known to be BQP-complete. In fact, it is likely that this problem is significantly weaker, in the computational complexity sense. Shor and Jordan [36] recently showed that the trace closure problem is complete for the one-clean qubit model of quantum computation, a model which is believed to be strictly weaker than standard quantum computation. We discuss the difference between the trace and plat closure cases in Sect. 1.4.

1.3 Description of the Algorithm

The essence of the algorithm lies in the fact that for braids with n strands, the pictures that one gets by replacing each crossing by one of the two pictures $\{$  ,  $\}$, can be assigned the structure of an algebra. This algebra is called the Temperley Lieb algebra, and denoted by TL_n. In fact, the map from the crossing to the appropriate linear combination of the above two pictures defines a *representation* of the group B_n of braids of n strands, inside the TL_n algebra. The Jones polynomial of the trace closure of a braid can be seen as a certain trace function (i.e., a linear function satisfying $\mathrm{tr}(AB) = \mathrm{tr}(BA)$) on the image of the braid in the TL_n algebra. Thus, using this language, the problem of approximating the Jones polynomial can be viewed as the problem of approximating the trace function of a certain element in TL_n.

Our goal is then to design an algorithm that approximates this trace. To this end we use an important fact about this trace: it satisfies an additional property called the *Markov property*. Moreover, this property makes it unique; any trace function on the TL_n algebra (or a representation of it) that satisfies this property is equal to the above trace! This leads us to the key idea of the algorithm: suppose we can define a representation of the TL_n algebra by matrices operating on qubits, and we can identify and estimate the trace that satisfies the Markov property on this representation. Then by the uniqueness of this trace we can estimate the Jones polynomial.

But what is the representation that should be used? If our intent is to design a quantum algorithm, it is best if the representation induced on the braid group be unitary, so that we can hope to approximate its trace by a quantum computer. Fortunately, it is in fact possible to give representations of the Temperley Lieb algebra which induce *unitary* representations of the braid group. Such representations were implicit in [22, 24] and explicitly stated in [18], among other places. They are called the *path model* representations. If we want to evaluate the Jones polynomial $V_L(t)$ for L a closure of a braid in B_n, and $t = e^{2\pi i/k}$, we use the kth path model representation of B_n. It is fairly straightforward to adapt these representations to work on the space of n qubits. Moreover, it is easy to show that the resulting unitary matrices which are the images of the generators of the braid group under the path model representation, can be applied efficiently by a quantum computer. To apply the image of the entire braid, we simply apply the images of the generators constituting the braid, one by one. We find that the image (by the path model representation) of an entire braid B can be applied efficiently by a quantum computer. Let us call the unitary matrix corresponding to a braid $Q(B)$.

To approximate the Jones polynomial of a trace closure of the braid, it suffices to approximate the Markov trace of $Q(B)$. This is done using standard quantum and classical algorithmic techniques, including the well known Hadamard test. The algorithm for the plat closure builds on similar ideas, though it is not directly stated in terms of traces. Thus we obtain a polynomial quantum algorithm for the BQP-complete problem of approximating the Jones polynomial of a plat closure of a braid.

1.4 Related Work and Further Directions

We have provided a simple algorithm for a BQP-complete problem, which is very different in its methods from previous quantum algorithms. In essence, the algo-

rithm isolates a certain local combinatorial structure of the problem, and assigns gates which somehow exhibit the same local structure. The gates are chosen by first considering an algebra whose relations capture the combinatorics of the problem - and then considering unitary representations of this algebra. We note that this general scheme was already used in the work of Kauffman and Lomonaco [27] who studied the problem of computing the Jones polynomial when the number of strands in the braid is equal to 3. This case can be solved efficiently classically, and so there is no advantage to using a quantum computer here, but it is never the less illuminating: unitary representations appear there not just in a discrete set of values, but in fact for the entire range $A = e^{i\theta}$ for $|\theta| \leq \pi/6$ and $|\theta - \pi| \leq \pi/6$. A much more detailed description of this case appeared recently in [28], following the publication of the preliminary version of this paper.

Our main hope for this work has been that this more combinatorial direction in quantum algorithms would lead to further discovery of quantum algorithmic speedups. Indeed, since the first publication of these results in [3], several generalizations and extensions have already been discovered. First, the algorithm has been further explained by Lomonaco and Kauffman in [30], and generalized to other closures beyond the plat and trace closures of a braid by Wocjan and Yard [40], where other complexity results related to the algorithm have also been derived. The results have been further generalized by several groups: Garnerone, Marzouli and Rasseti provided efficient algorithms for the colored Jones polynomial (discussed in several papers [14–17]), and Kauffman and Lomonaco [26], have given algorithms for the colored Jones polynomial and for another topological invariant. Generalizing the results to other topological invariants is left for further research.

Of particular interest is the recent work of Aharonov, Arad, Eban and Landau [2]. Though not its focus, the work in [2] provides a significant simplification of the work presented here. This is due to the fact that [2] use a generalized version of the Temperley-Lieb Algebras, that allows an algebra element to have different numbers of in-going and out-going strands. The first implication is that the entire discussion of the notion of Markov trace and its uniqueness (Sect. 2.12), is no longer needed; this significantly simplifies the analysis of our algorithms (Sect. 3.3).

More importantly, the main achievement in [2] is the ability to use *non-unitary* representations of the Temperley-Lieb algebras. The current paper is restricted to unitary representations, and is therefore limited to the very special cases (namely, values of t) where such unitary representations exist. All previous generalizations of the algorithm mentioned above were also restricted to unitary representations, as it seems like a necessary requirement in the context of quantum computation. In [2] the unitarity constraint is relaxed, which allows a generalization of the results presented here to all points of the Tutte polynomial, of which the Jones polynomial is a special case.

It is not immediately clear that these quantum algorithms perform a difficult or even a non-trivial task. Perhaps similar additive approximations are doable in polynomial time by classical computers? or maybe the approximations are so crude that they are simply trivial? However, the results of [2] give strong evidence that this is not the case. It is shown that the approximations of the Jones polynomial at $e^{2\pi i/k}$ for constant values of k are BQP-hard, namely, they cannot be achieved in classical

polynomial time unless quantum polynomial time is equal to classical (probabilistic) polynomial time (which is hard to believe). Following the preliminary version of the current paper, [2] extended the BQP-hardness results to non-constant k's, as long as they are polynomially bounded. In [2], BQP-hardness was shown for a large subset of new parameters, providing a whole new class of BQP-complete problems beyond the Jones polynomial approximations provided here. Importantly, many of these new quantum-universal problems correspond to parameters for which the Temperley-Lieb algebra representations are non-unitary. The fact that non-unitary representations can encode universal quantum systems seems likely to have implications outside the area of quantum computation, but this is yet to be discovered.

One of the major goals of this work, as is stated in its preliminary version [3], was to initialize progress on the important problem of approximating the Q-state Potts model partition function. This problem has been suggested as a candidate for efficient quantum algorithms more than a decade ago, but, unfortunately, no significant progress has been made since then. The new algorithms of [2] extending this work, apply for the entire Tutte polynomial, of which the Q-state Potts model partition function is a special case; they thus provide efficient quantum additive approximations for the Potts model. We note that, unfortunately, the universality proofs of [2] do not hold for the Potts model parameters, which means that it is not clear that these new algorithms for the Potts model achieve tasks which are not doable classically. We hope that further work will clarify the situation.

A particularly intriguing possible direction to pursue is to try and use (unitary or non-unitary) representations of algebras other than the Temperley-Lieb algebras to encode combinatorial structures related to other combinatorial problems. We believe that further implications and extensions are awaiting down the road.

We remark that after completing this work, we learned about a previous independent attempt to prove similar results [34] for the Jones polynomial, using unitary representations of the braid group. Unfortunately, the work of [34] is greatly flawed, and in particular claims to provide an exact solution to the $\#P$-hard problem.

A curious question that emerges from this work relates to the difference between the plat and the trace closures problems. It is known that any plat closure of a braid can be transformed efficiently into a trace closure of some other braid [37]. The reader might therefore find it curious that one of these problems is BQP-complete, while the other one is not known to be so. The explanation lies in the fact that the quality of the approximation in both algorithms depends *exponentially* on the number of strands in the braid. The transformation from plat to trace closures requires, in the worst case, a significant increase in the number of strands. This, unfortunately, degrades the quality of the approximation exponentially. Following the publication of the first version of this paper, it was recently shown by Shor and Jordan [36] that the problem of approximating the Jones polynomial of the trace closure of a braid, at the fifth root of unity, is complete for a very restricted model of quantum computation, known as the one clean qubit model. This model is believed, though not proved, to be strictly less powerful than the standard quantum computation model.

To conclude, we believe that this paper helps to clarify and de-mystify (at least one direction of) the intriguing equivalence between quantum computation and the problem of approximating the Jones polynomial, and moreover, opens up the path to new

quantum algorithms based on combinatorial structures encoded using algebra representations. We hope this leads to a deeper understanding of quantum computation complexity.

Organization of Paper Section 2 provides the background and the necessary definitions, starting from quantum computation, the Hadamard test, the braid group, algebras and representations, the Jones polynomial, the Temperley Lieb algebras and the path model representation. Using these notions we describe the algorithms in Sect. 3 and prove their correctness.

2 Background

2.1 Efficient (Quantum or Classical) Algorithms

We use the term "efficient algorithm" in its usual computer science sense, namely, the number of elementary computational steps that the algorithm applies is polynomial in the length of the input. The length of the input is the number of bits in the string that describes the input. As we will see, the input for the algorithm to be discussed in this paper consists of a pair: a braid of n strands and m crossings, and an integer k. One of course has to specify how to encode the input braid by a bit string. This can be done, for example, by providing the braid as a product of generators, and specifying the indices of the generators according to the order they are applied. This gives a description whose length is polynomial in n and m. The exact way to do this does not matter. With respect to the parameter k, the term "efficient" in this paper means polynomial in k (rather than in $\log(k)$, the length of the bit representation of k). For consistency with the previous discussion, one can think of k as being input in unary representation.

2.2 Quantum Computation

For background on quantum computation, see [31]. We mention here several basic facts. Strictly speaking, we use here a quantum-classical hybrid model of computation, in which a classical probabilistic Turing machine performs calls to a quantum computer, and uses its outcomes to perform some classical computations. It is standard that this model can be simulated efficiently by the standard quantum computation model. The term BQP refers to the class of problems solvable in polynomial time by a quantum computer, with error probability less than a $1/3$ for both yes and no instances. A problem L is BQP-hard if a solution to it implies a solution to all of BQP: more formally, given a problem L' in BQP, there exists a function (computable in classical polynomial time) which maps yes (no) instances of L' to yes (no) instances of L. A problem is BQP-complete if it is both in BQP and is BQP-hard.

2.3 The Hadamard Test

The following fact is standard in quantum computation. If a state $|\alpha\rangle$ can be generated efficiently, and a unitary Q can be applied efficiently, then there exists an efficient

quantum circuit whose output is a random variable $\in \{-1, 1\}$, and whose expectation is $\mathcal{R}e\langle\alpha|Q|\alpha\rangle$. Start with the two-register state $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |\alpha\rangle$, apply Q conditioned on the first qubit to get the state $\frac{1}{\sqrt{2}}(|0\rangle \otimes |\alpha\rangle + |1\rangle \otimes Q|\alpha\rangle$, apply a Hadamard gate on the first qubit, and measure. Output 1 if the measurement result is $|0\rangle$, -1 if the measurement result is $|1\rangle$. The expectation of the output is exactly $\mathcal{R}e\langle\alpha|Q|\alpha\rangle$. To get a random variable whose expectation is the imaginary part, start with the state $\frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) \otimes |\alpha\rangle$ instead.

2.4 Algebra Background

An algebra is a vector space with a multiplication. The multiplication must be associative and distributive. A representation of a group G inside an algebra is a group homomorphism ρ from G to the group of invertible elements in the algebra, namely, we require $\rho(g_1)\rho(g_2) = \rho(g_1 g_2)$ for any $g_1, g_2 \in G$.

We shall sometimes refer to a representation of G without specifying the algebra; in these cases we shall mean a representation inside the algebra of $n \times n$ matrices. We shall be interested in algebra representations as well:

Definition 2.1 An r dimensional representation Φ of an algebra is a linear mapping from the algebra into the set of $r \times r$ complex matrices M_r, such that for any two elements X, Y in the algebra, $\Phi(X)\Phi(Y) = \Phi(XY)$.

If a group is represented inside an algebra then any representation of the algebra gives a representation of the group by composition.

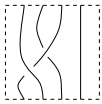
Often, an algebra or a group is defined using a set of generators and relations between them. In this case, a representation may be defined by specifying the images of the generators, provided the same relations hold for the images as for the generators.

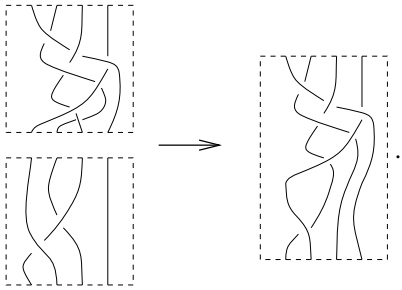
2.5 The Braid Group

Consider two horizontal bars, one on top of the other, with n pegs on each. By an *n strand braid* we shall mean a set of n strands such that: (1) Each strand is tied to one peg on the top bar and one peg on the bottom bar, (2) Every peg has exactly one end attached to it, (3) The strands may pass over and under each other, (4) The tangent vector of every strand at any point along the path from top to bottom always has a non-zero component in the downward direction. Here is an example of a 4-strand braid:



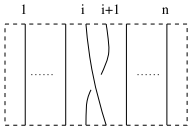
The set of n-strand braids, B_n, has a group structure with multiplication as follows. Given two n-strand braids b_1, b_2, place braid b_1 above b_2, remove the bottom b_1 bar and the top b_2 bar and fuse the bottom of the b_1 strands to the top of the b_2 strands.

The product of the above 4-strand braid with the 4-strand braid  is:



An algebraic presentation of the braid group due to Artin is as follows [4]: Let B_n be the group with generators $\{1, \sigma_1, \ldots, \sigma_{n-1}\}$ and relations

1. $\sigma_i \sigma_j = \sigma_j \sigma_i$ for $|i - j| \geq 2$,
2. $\sigma_i \sigma_{i+1} \sigma_i = \sigma_{i+1} \sigma_i \sigma_{i+1}$.

This algebraic description corresponds to the pictorial picture of braids: σ_i corresponds to the pictorial braid 

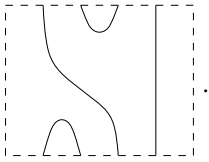
, and concatenating such pictures gives a general braid in B_n.

2.6 The Temperley-Lieb Algebras

Definition 2.2 Given n an integer and d a complex number we define the Temperley-Lieb algebra $TL_n(d)$ to be the algebra generated by $\{1, E_1, \ldots, E_{n-1}\}$ with relations

1. $E_i E_j = E_j E_i$, $|i - j| \geq 2$,
2. $E_i E_{i \pm 1} E_i = E_i$,
3. $E_i^2 = d E_i$.

There is a well known geometric description of $TL_n(d)$ due to Kauffman [25]. We denote it by $gTL_n(d)$. It uses the notion of Kauffman n-diagrams, which is best explained by an example, e.g., a Kauffman 4-diagram:

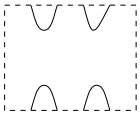


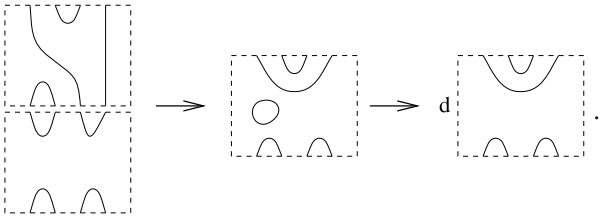
In general, a Kauffman n-diagram is a diagram as above, with n top pegs and n bottom pegs, and no crossings and no loops. More formally:

Definition 2.3 Let D_n be a rectangle with n marked points on the top of the boundary and n marked points on the bottom. A *Kauffman n-diagram* is a picture sitting inside D_n consisting of n non-intersecting curves that begin and end at distinct marked boundary points. We will consider two such diagrams equal if they are isotopically equivalent (keeping the boundary fixed).

We define a vector space over these diagrams:

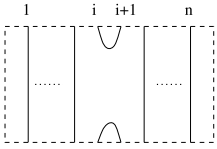
Definition 2.4 Let $\mathcal{K}_n$ be the vector space of complex linear combinations of Kauffman n-diagrams.

Multiplication of two Kauffman n-diagrams is done just like in the case of braids. To multiply a diagram k_1 with a diagram k_2 we stack k_1 on top of k_2, and fuse the matching ends of the strands. The resulting diagram is a new Kauffman diagram with possibly some extra closed loops. Let m be the number of these extra loops, and let k_3 be the new diagram with those extra loops removed. We define the product of k_1 and k_2 to be $d^m k_3 \in \mathcal{K}_n$. For example, a multiplication of the above Kauffman 4-diagram with the Kauffman 4-diagram  results in:



This multiplication rule can be extended linearly to $\mathcal{K}_n$, we call the resulting algebra $gTL_n(d)$.

The algebras $TL_n(d)$ and $gTL_n(d)$ are isomorphic:

Theorem 2.5 *The map* $\psi : TL_n(d) \to gTL_n(d)$ *given by the homomorphic extension of* $\psi(E_i) =$ 

is an isomorphism.

Proof It is a simple and fun exercise to check that the image of the relations given in Definition 2.2 are relations in $gTL_n(d)$. For the remaining details see [5]. □

We shall refer to the pictures of the form  , which generate $gTL_n(d)$, as *capcups*.

2.7 Representing B_n Inside $TL_n(d)$

We define a mapping from the braid group to $TL_n(d)$:

Definition 2.6 $\rho_A : B_n \mapsto TL_n(d)$ is defined by

$$\rho_A(\sigma_i) = AE_i + A^{-1}I, \qquad \rho_A(\sigma_i^{-1}) = A^{-1}E_i + AI.$$

Claim 2.7 *For a complex number A which satisfies $d = -A^2 - A^{-2}$, the mapping ρ_A is a representation of the braid group B_n inside $TL_n(d)$.*

Proof We need to check that the relations of the braid group are satisfied by this mapping. For $|i - j| > 1$, $\rho_A(\sigma_i)$ commutes with $\rho_A(\sigma_j)$ since E_i commutes with E_j. To show that $\rho_A(\sigma_i)\rho_A(\sigma_{i+1})\rho_A(\sigma_i) = \rho_A(\sigma_{i+1})\rho_A(\sigma_i)\rho_A(\sigma_{i+1})$, substitute to get an expression in E_i's. Opening up the first expression we get $A^3E_iE_{i+1}E_i + AE_{i+1}E_i + AE_i^2 + A^{-1}E_i + AE_iE_{i+1} + A^{-1}E_{i+1} + A^{-1}E_i + A^{-3}$. The second expression gives $A^3E_{i+1}E_iE_{i+1} + AE_iE_{i+1} + AE_{i+1}^2 + A^{-1}E_{i+1} + AE_{i+1}E_i + A^{-1}E_i + A^{-1}E_{i+1} + A^{-3}$. We remove similar terms, and using the relations of the $TL_n(d)$ it remains to show that $(A^{-1} + Ad + A^3)E_i = (A^{-1} + Ad + A^3)E_{i+1}$. This holds because the constants are 0 due to the relation between d and A. □

2.8 Unitary Representation of B_n

Given a representation τ of $TL_n(d)$, we may use the representation of the braid group inside the $TL_n(d)$ algebra (Definition 2.6) to derive a representation of B_n by composition, as follows. Define the map φ by specifying its operation on the generators σ_i of B_n to be $\varphi(\sigma_i) = \varphi_i = \tau(\rho_A(\sigma_i)) = A\tau(E_i) + A^{-1}I$. This representation is unitary under certain constraints:

Claim 2.8 *If $|A| = 1$ and $\tau(E_i)$ are Hermitian for all i, then the map φ is a unitary representation of B_n.*

Proof $\tau(\rho_A(\sigma_i))\tau(\rho_A(\sigma_i))^\dagger = (A^{-1}I + A\tau(E_i))((A^{-1})^*I + A^*\tau(E_i)^\dagger) = I + A^{-2}\tau(E_i) + A^2\tau(E_i) + d\tau(E_i) = I$. □

2.9 Tangles

For this paper, we define a tangle to be a braid in which some of its crossings have been replaced by a picture of the form $\{\cup\ \cap\}$. Braids and Kauffman diagrams are tangles.

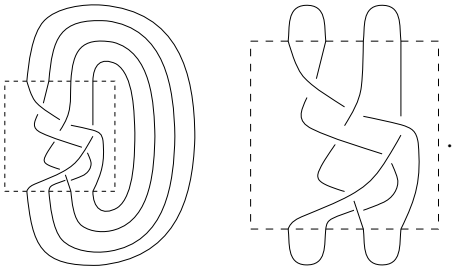
2.10 From Braids to Links

We can connect up the endpoints of a braid in a variety of ways to get links. We single out two such ways:

Definition 2.9 The *trace closure* of a braid B shall be the link achieved by connecting the strand at the rightmost top peg, around to the right, to the strand at the rightmost bottom peg, then connecting in the same way the next two rightmost top and bottom strands and so on. We denote the resulting link by B^{tr}.

Definition 2.10 The *plat closure* of a $2n$-strand braid shall be the link formed by connecting pairs of adjacent strands (beginning at the leftmost strand), on both the top and bottom. We denote the resulting link by B^{pl}.

Examples of the trace closure and the plat closure of the same 4-strand braid are:



These closures are also well defined for tangles.

2.11 The Jones Polynomial

A definition of the Jones polynomial $V_L(t)$ due to Kauffman [25] is as follows. We start by defining the Kauffman bracket $\langle L\rangle$ as a polynomial in A for A such that $A^{-4} = t$.

Definition 2.11 Consider a link L, given by a link diagram. A state σ of L shall mean a choice, at each crossing  of L, from the set $\{$  ,  $\}$. To a state σ of a link L we associate the following expression $\sigma(L)$: Let σ^+ (respectively σ^-) be the number of crossings for which σ chooses  (respectively ). Let $|\sigma|$ be the number of closed loops in the diagram gotten by replacing each crossing  by the choice indicated by the state σ. Define $\sigma(L) = A^{\sigma^+ - \sigma^-} d^{|\sigma|-1}$. The Kauffman bracket polynomial, also called the bracket state sum, for L, is defined to be

$$\langle L\rangle = \sum_{\text{all states } \sigma} \sigma(L).$$

To define the Jones polynomial, we consider oriented links, namely links with one arrow on each connected component. The connection between the Jones polynomial and the Kauffman bracket is given by a notion called the *writhe*:

Definition 2.12 For an oriented link L, assign to each crossing that looks like this  the value $+1$, and to each crossing that looks like this:  the value -1. The writhe of L is the sum over all the crossings of these signs.

Definition 2.13 The Jones polynomial of an oriented link L is defined to be

$$V_L(t) = V_L(A^{-4}) = (-A)^{-3w(L)} \cdot \langle L \rangle$$

where $w(L)$ is the writhe of the oriented link L, and $\langle L \rangle$ is the bracket state sum of the link L, ignoring the orientation.

Thus, the Jones polynomial is a scaled version of the bracket polynomial. Moreover, the writhe of a link can be easily calculated from the link diagram, and hence the problem of calculating the bracket sum polynomial is equivalent in complexity to that of calculating the Jones polynomial.

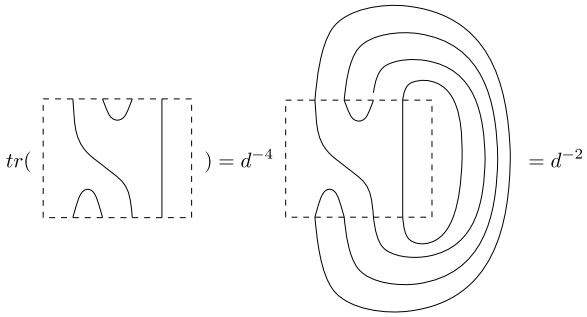
2.12 The Markov Trace

Definition 2.14 A linear function from an algebra to the complex numbers is called a *trace* if it satisfies $\mathrm{tr}(XY) = \mathrm{tr}(YX)$ for every two elements X, Y in the algebra.

We define the following trace on $gTL_n(d)$.

Definition 2.15 The *Markov trace* $\mathrm{tr} : gTL_n(d) \to \mathbb{C}$ is defined on a Kauffman n-diagram K as follows. Connect the top n labeled points to the bottom n labeled points of K with non-intersecting curves, as in the trace closure. Let a be the number of loops of the resulting diagram. Define $\mathrm{tr}(K) = d^{a-n}$. Extend tr to all of $gTL_n(d)$ by linearity.

For example:



Since $TL_n(d)$ and $gTL_n(d)$ are isomorphic, tr induces a trace on $TL_n(d)$; for simplicity we shall denote this map by tr as well. We note that $\mathrm{tr}(X)$ does not depend on whether X is considered as an element of $TL_n(d)$ or of $TL_{n'}(d)$ for some $n' > n$, by adding trivial strands to the right.

Claim 2.16 tr *satisfies the following three properties*:

1. $\mathrm{tr}(1) = 1$,
2. $\mathrm{tr}(XY) = \mathrm{tr}(YX)$ *for any* $X, Y \in TL_n(d)$,
3. *If* $X \in TL_{n-1}(d)$ *then* $\mathrm{tr}(XE_{n-1}) = \frac{1}{d}\mathrm{tr}(X)$.

Proof It is straightforward to verify this by examining the appropriate pictures in $gTL_n(d)$. □

Of particular importance is the third property, which is referred to as the *Markov* property. These three properties uniquely determine a linear map on $TL_n(d)$:

Lemma 2.17 [23] *There is a unique linear function* tr *on* $TL_n(d)$ (*and on any representation of it*) *that satisfies properties* 1–3.

Proof By a reduced word $w \in TL_n(d)$ we shall mean a word in the set $\{1, E_1, \ldots, E_{n-1}\}$ that is not equal to cw' for any c a constant and w' a word of smaller length. Using the relations of $TL_n(d)$ and applying simple combinatorial arguments, we show that a reduced word $w \in TL_n(d)$ contains at most one E_{n-1} term. We induct on n. Clearly the only reduced words in $TL_2(d)$ are 1 and E_1. Assume the statement is true for reduced words in $TL_{n-1}(d)$. Suppose there exists a reduced word $w \in TL_n(d)$ containing more than one E_{n-1} term. Write $w = w_1E_{n-1}w_2E_{n-1}w_3$ with w_2 a word without E_{n-1}. Since w_2 must be reduced and is in $TL_{n-1}(d)$, the induction hypothesis implies w_2 contains at most one E_{n-2} term. If w_2 does not contain a E_{n-2} term, $w_2 \in TL_{n-2}(d)$ and it commutes with E_{n-1} so we have $w = w_1w_2E_{n-1}E_{n-1}w_3$ which shows that w was not reduced. Otherwise we can write $w_2 = vE_{n-2}v'$ with v, v' both words in $TL_{n-2}(d)$. It follows therefore that v and v' commute with E_{n-1} and thus $w = w_1vE_{n-1}E_{n-2}E_{n-1}v'w_3$ which again shows that w was not reduced. We conclude that any reduced word in $TL_n(d)$ contains at most one E_{n-1} term.

Given $w \in TL_n(d)\backslash TL_{n-1}(d)$ a reduced word we write $w = w_1E_{n-1}w_2$ with w_1, $w_2 \in TL_{n-1}(d)$. Then $\mathrm{tr}(w) = \mathrm{tr}(w_2w_1E_{n-1}) = d\,\mathrm{tr}(w_2w_1)$, the first equality by property 2, the second by property 3. Thus for any word $w \in TL_n(d)$ we can reduce the trace computation to the trace of a word $w_2w_1 \in TL_{n-1}(d)$. Iterating this process, (and using the fact that $\mathrm{tr}(1) = 1$), we see that the trace of a word in $TL_n(d)$ is uniquely determined by the relations 1–3. Since the trace is linear, the result follows. □

We have the following convenient description of the Jones polynomial in terms of the Markov trace.

Lemma 2.18 *Given a braid* B, *then*

$$V_{B^{\mathrm{tr}}}(A^{-4}) = (-A)^{-3w(B^{\mathrm{tr}})}d^{n-1}\,\mathrm{tr}(\rho_A(B)).$$

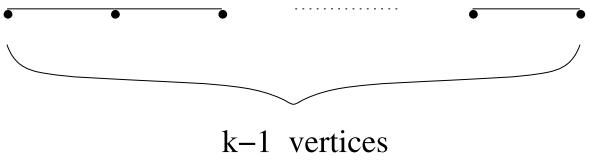
Proof By Definition 2.13, we need to show that $\langle B^{\mathrm{tr}}\rangle = \mathrm{tr}(\rho_A(B))d^{n-1}$. We observe that there exists a one to one correspondence between states that appear in the bracket

sum $\langle B^{\mathrm{tr}} \rangle$, and Kauffman n-diagrams that appear in $\rho_A(B)$. The weight of an element in the bracket state sum corresponding to the state σ is $A^{\sigma^+ - \sigma^-} d^{|\sigma|-1}$. We observe that the corresponding Kauffman n-diagram appears in $\rho_A(B)$ with the weight $A^{\sigma^+ - \sigma^-}$. Hence, by linearity of the trace, it remains to show that for each σ, the trace of the Kauffman diagram corresponding to σ, times d^{n-1}, equals to the remaining factor in the contribution of σ to the bracket state sum, $d^{|\sigma|-1}$. This is true since by the definition of the trace of a Kauffman diagram, it is exactly $d^{|\sigma|-n}$. □

This lemma also holds if B is replaced by a tangle.

2.13 The Path Model Representation of $TL_n(d)$

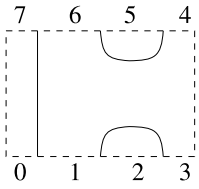
We describe the path model representation of $TL_n(d)$ implicit in [22, 24] (an explicit description is given in [18], among other places). The representation will act on a vector space determined by paths on a graph. Specifically, given an integer k (k will be chosen in relation to d later), let G_k be the straight line graph with $k-2$ segments and $k-1$ vertices:



Define $Q_{n,k}$ to be the set of all paths of length n on the graph G_k beginning at the leftmost vertex. Given $q \in Q_{n,k}$, we shall denote by $q(0), q(1), \ldots, q(n)$ the sequence of vertices of G_k describing q; thus q(0) is the leftmost vertex and $q(i)$ and $q(i+1)$ are adjacent vertices of G_k for all i. We shall think of the elements of $Q_{n,k}$ as an orthonormal basis of a vector space $\mathcal{V}_{n,k}$; hence an element $q \in Q_{n,k}$ shall represent both a path on G_k and a basis element of $\mathcal{V}_{n,k}$. We now construct the path model representation $\tau(TL_n(d)) : \mathcal{V}_{n,k} \to \mathcal{V}_{n,k}$.

Given a Kauffman n-diagram T, to describe $\tau(T)$ it will suffice to give the matrix entry $\tau(T)_{q',q}$ for each pair $q', q \in Q_{n,k}$. To do this, we note that the strands of a Kauffman diagram separate the rectangle into regions; we would like to label the regions by vertices of G_k, such that the labeling of the bottom part of T will correspond to q and the top part to q', and then compute the matrix element $\tau(T)_{q',q}$ from the labels. This is done as follows.

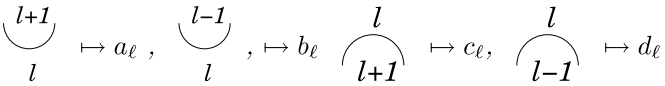
The n marked points of a Kauffman n-diagram divide the top and bottom boundary into $n+1$ segments which we shall refer to as *gaps*. We shall say a set of gaps that bound the same region in the diagram are *connected*. For example, in the following Kauffman 3-diagram:



the set of gaps $\{0, 7\}$ are connected as are the set of gaps $\{1, 3, 4, 6\}$. We shall say that the pair (q', q) is *compatible* with T if once we label the gaps on the bottom from left to right by $q(0), q(1), \ldots, q(n)$ and we label the gaps on the top from left to right by $q'(0), q'(1), \ldots, q'(n)$, then any set of connected gaps are all labeled by the same vertex of G_k. Thus in this case each region of T can be thought of as being labeled by a single vertex of G_k. Moreover, we require that adjacent regions are labeled by adjacent labels of the graph.

The matrix entry $\tau(T)_{q',q}$ will only be nonzero in case the pair of paths (q', q) is *compatible* with T. In this case, the regions are indeed labeled by vertices in G_k; we can now do the following. To each local maximum and minimum of the Kauffman diagram T, we associate a complex number that depends on the labeling of the regions that surround them as follows:

Definition 2.19 The coefficients $a_\ell, b_\ell, c_\ell, d_\ell$. For $\ell \in \{1, \ldots, k-1\}$, we associate the following coefficient with a maxima or a minima whose regions are labeled:



This holds whenever both labels are sites of G_k, namely, are integers in $\{1, \ldots, k-1\}$. Note that because of this requirement, a_{k-1}, c_{k-1}, d_1 and b_1 have not been associated any labeled diagram. We set those four coefficients to zero.

The matrix element $\tau(T)_{q',q}$ at a compatible pair (q', q), is defined to be the product of the appropriate complex numbers over all local maxima and minima in T.

For the map $\tau(T)$ described above to be well-defined, it has to give the same result for isotopic Kauffman diagrams. An isotopic move can be seen to only create or eliminate local maxima and minima in pairs; we see that the conditions

$$b_{\ell+1}c_\ell = 1 = a_\ell d_{\ell+1} \quad \forall \ell \in \{1, \ldots, k-2\} \tag{1}$$

are necessary and sufficient for the map to be isotopically invariant. A single extra constraint is needed to produce a representation of $TL_n(d)$:

Claim 2.20 *If the coefficients $a_\ell, b_\ell, c_\ell, d_\ell$ satisfy* (1) *and*

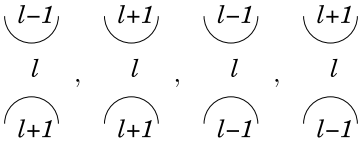
$$d = b_\ell d_\ell + a_\ell c_\ell, \tag{2}$$

for all $\ell \in \{1, \ldots, k-1\}$ then the resulting map τ defined as above is a representation of $TL_n(d)$.

Proof To prove the result we need only verify that the matrices $\tau(E_i)$ satisfy the relations of Definition 2.2. This amounts to verifying that the matrix elements of the operators on both sides of each relation are equal. Pictorially, a matrix element in a product of operators is given by stacking the operators together, and summing up over all possible labeling inside loops, such that the label inside the loop is different

by exactly one from the label outside the loop. This summation corresponds to the summation over the intermediate index in matrix multiplication.

We now check for the different relations in Definition 2.2. For the generator E_i, there are only four types of non-zero elements, namely, four types of compatible pairs corresponding to the following types of labeling of the regions near the i-th strand:



In the first two relations no loops are created when the operators are multiplied, so the verification follows from the isotopy invariance of τ (i.e., (1)). The third relation follows from (2), using the fact that one loop was created and there are two possible ways to label the region inside. □

We would like $\tau(E_i)$ to be Hermitian, so that the induced representation on B_n is unitary by Claim 2.8. For this we add the constraints

$$a_\ell = c_\ell^*, b_\ell = d_\ell^* \tag{3}$$

Claim 2.21 *If the coefficients $a_\ell, b_\ell, c_\ell, d_\ell$ satisfy (1)–(3) then $\tau(E_i) = \tau(E_i)^\dagger$ for all i.*

Proof We need to prove that $\tau(E_i)_{q,q'} = (\tau(E_i)_{q',q})^*$. We need to check only for compatible pairs, namely for q, q' which on the $i-1, i, i+1$ entry are equal to $\ell, \ell \pm 1, \ell$ for some ℓ. This amounts to checking that $a_\ell d_\ell = b_\ell^* c_\ell^*$, $b_\ell d_\ell = b_\ell^* d_\ell^*$, and $a_\ell c_\ell = a_\ell^* c_\ell^*$. This follows from (3). □

It is now left to solve (1)–(3) to derive the definition of τ.

Claim 2.22 *Define $\lambda_\ell = \sin(\pi\ell/k)$ for $\ell \in \{1, \ldots, k-1\}$. Then $a_\ell = c_\ell^* = \sqrt{\frac{\lambda_{\ell+1}}{\lambda_\ell}}$, $b_\ell = d_\ell^* = \sqrt{\frac{\lambda_{\ell-1}}{\lambda_\ell}}$ satisfy (1)–(3), with $d = 2cos(\pi/k)$.*

Proof It is easy to check that the above coefficients satisfy the requirements, using simple trigonometric identities. We provide the detailed derivation of the solution to the equations, since it is insightful to see its connection to the adjacency matrix of G_k and its principle eigenvector. Define $x_\ell = |a_\ell|^2$, for $\ell \in \{1, \ldots, k-1\}$, and $x_0 = 1$ (any arbitrary constant would work, as x_0 will serve as a normalizing constant). We solve for x_ℓ for $\ell \in \{1, \ldots, k-1\}$. We have the following set of equations: $d = x_1$, and for $\ell \in \{2, \ldots, k-1\}$, $\frac{1}{x_{\ell-1}} + x_\ell = d$. We define $y_\ell = x_0 \cdots x_\ell$ for $\ell \in \{0, \ldots, k-1\}$, and we set $y_{-1} = 0$. We get the following equation for y_ℓ: $y_{\ell-2} + y_\ell = d y_{\ell-1}$ for $\ell \in \{1, \ldots, k-1\}$, with $y_{-1} = y_{k-1} = 0$. These constraints are exactly those defining

the eigenvector $(y_0, \ldots, y_{k-2})$ of the $k-1 \times k-1$ adjacency matrix of the graph G_k

$$\begin{pmatrix} 0 & 1 & \cdots & & & 0 \\ 1 & 0 & 1 & & & \\ & 1 & & & & \\ \vdots & & & \ddots & & \\ & & & & 0 & 1 \\ 0 & & \cdots & & 1 & 0 \end{pmatrix},$$

with eigenvalue d. It is not difficult to check that $y_\ell = \lambda_{\ell+1}$ is exactly such an eigenvector. Since all coefficients are non-negative, we can solve for a_ℓ using $|a_\ell|^2 = x_\ell = y_\ell / y_{\ell-1}$. Equations (1) and (3) determine b_ℓ, c_ℓ, d_ℓ. □

Using these coefficients, we get the definition of $\tau(E_i)$ as follows. $\tau(E_i)_{q,q'} = 0$ if (q, q') is not compatible with E_i. For a compatible pair (q, q'), $\tau(E_i)_{q,q'}$ is the product of two coefficients, one corresponding to the maximum and the other to the minimum in E_i. For example, if $q = q'$ both move from the site ℓ to $\ell + 1$ in the ith step, and then return to the site ℓ in the $i + 1$th step, we have $\tau(E_i)_{q,q} = a_\ell c_\ell$. Given $\tau(E_i)$, we can extend our definition of τ to all elements in $TL_n(d)$.

2.14 Unitary Path Model Representation of B_n

The previous section provided a representation τ of $TL_n(d)$ defined for $d = 2cos(\pi/k)$. We have that $d = -A^2 - A^{-2}$ for $A = ie^{-\pi i/2k}$, and also that $\tau(E_i)$ are Hermitian. Thus, the conditions of Sect. 2.8 are satisfied. We define:

Definition 2.23 The unitary path model representation of B_n is defined to be $\varphi(B) = \tau(\rho_A(B))$.

The map φ can be extended to operate on tangles by letting ρ_A be applied only to the crossings in the tangle.

3 The Quantum Algorithm

We are now ready to prove Theorems 1.1 and 1.2. We first translate the path model representation to work on qubits, and show that it can be implemented efficiently. We use this to design the algorithms, and then prove their correctness.

3.1 Moving to Qubits

The adaptation of the path model representation to qubits is fairly straightforward. We simply switch from presenting paths by the list of their locations, to a binary representation which indicates the *direction* of each step. Thus, we shall interpret a string of n bits to be a sequence of instructions, where a 0 shall mean take one step to the left and a 1 shall mean take one step to the right. We shall restrict our attention

to those n bit strings that describe a path that starts at the leftmost vertex of G_k and remains inside G_k at each step. From here on when we say "path" we actually mean the bit string that represents the path.

Definition 3.1 We define $P_{n,k,\ell}$ to be the set of all paths p on G_k of n steps which start at the left most site and end at the ℓ's site. We define the subspace $\mathcal{H}_{n,k,\ell}$ to be the span of $|i\rangle$ over all $i \in P_{n,k,\ell}$. In a similar way, we define $P_{n,k}$ to be all paths with no restriction on the final point, i.e., $P_{n,k} = \bigcup_{l=1}^{k-1} P_{n,k,l}$, and we define $\mathcal{H}_{n,k}$ to be the span of the corresponding computational basis states.

We define a representation Φ as a homomorphism from $TL_n(d)$ to matrices operating on $\mathcal{H}_{n,k}$. To define Φ it suffices to specify the images of the E_i's, $\Phi(E_i) = \Phi_i$. The operators Φ_i are defined so that they correspond to the operators $\tau(E_i)$ (see Sect. 2.13) via the natural isomorphism between $\mathcal{V}_{n,k}$ and $\mathcal{H}_{n,k}$. Thus, the transition from τ to Φ is merely a change of language. This is done as follows.

To uniquely define Φ_i on $\mathcal{H}_{n,k}$, it suffices to define what it does to each basis element, namely, to $|p\rangle$ for $p \in P_{n,k}$. We need the following notation:

Definition 3.2 Let $p|_i$ denote the restriction of a path p to its first $i-1$ coordinates. Given a path p on G_k, we denote by $\ell(p) \in \{1, \ldots, k-1\}$ the location in G_k that the path p reached in its final site. Denote $z_i = \ell(p|_i)$.

We can now define the operation of Φ_i. Φ_i is defined as an operation on the first $i+1$ coordinates in a path p:

$$\begin{aligned}
\Phi_i|p|_i 00\rangle &= 0,\\
\Phi_i|p|_i 01\rangle &= \frac{\lambda_{z_i-1}}{\lambda_{z_i}}|p|_i 01\rangle + \frac{\sqrt{\lambda_{z_i+1}\lambda_{z_i-1}}}{\lambda_{z_i}}|p|_i 10\rangle,\\
\Phi_i|p|_i 10\rangle &= \frac{\lambda_{z_i+1}}{\lambda_{z_i}}|p|_i 10\rangle + \frac{\sqrt{\lambda_{z_i+1}\lambda_{z_i-1}}}{\lambda_{z_i}}|p|_i 01\rangle,\\
\Phi_i|p|_i 11\rangle &= 0.
\end{aligned} \tag{4}$$

To apply Φ_i on the n-bit string $|p\rangle$ we tensor the above transformation with identity on the last $n-i-1$ qubits. For dealing with the edge cases, we use the convention $\lambda_j = 0$ for any $j \notin \{1, .., k-1\}$.

Once we have defined Φ_i, Φ is then extended to the entire algebra by the multiplication property of a representation, and by linearity.

Claim 3.3 *Φ is a representation of $TL_n(d)$. Φ_i are Hermitian. Moreover, Φ induces a unitary representation of the Braid group B_n, operating on $\mathcal{H}_{n,k}$.*

Proof The proof follows from the corresponding properties of τ, namely Sects. 2.13, 2.14, and the natural isomorphism between paths on G_k presented by their sequence of locations, $\mathcal{V}_{n,k}$, and paths presented as bit strings, namely $\mathcal{H}_{n,k}$. □

3.2 Efficient Application of One Crossing

With the risk of confusion, we denote the unitary representation of B_n induced by Φ, also by φ as in Definition 2.23. The only difference is that now $\varphi(B)$ operates on $\mathcal{H}_{n,k}$ rather than on $\mathcal{V}_{n,k}$. Denote φ_i to be the unitary image of the ith braid generator. The matrices φ_i are defined so far only on $\mathcal{H}_{n,k}$ which is a subspace of the Hilbert space of n qubits; we arbitrarily define their extension to the rest of the Hilbert space to be the identity.

Claim 3.4 *For all $i \in \{1, \ldots, n-1\}$, φ_i can be implemented on the Hilbert space of n qubits using* $\mathrm{poly}(n, k)$ *gates.*

Proof We note that the application of φ_i on $p \in P_{n,k}$ modifies only the $i, i+1$ bits of p, and the modification depends on the location up to the ith step, namely on $z_i = \ell(p|_{i-1})$. z_i is a number which can be calculated efficiently and written on $O(\log(k))$ ancilla qubits, using the following standard technique: Initialize a counter register of, say, $\log(2k)$ qubits to the value 1 (namely, to the string of $\log(2k) - 1$ zeroes followed by the bit 1). Then move along the qubits of the path p from left to right, and for each of the n qubits update the current state of the counter, ℓ, by applying

$$|b\rangle|\ell\rangle \mapsto |b\rangle|(\ell + (-1)^b) \bmod 2k\rangle$$

where b is the state of the currently read qubit. Since this is a unitary operation on $\log(2k) + 1$ qubits, it can be applied using polynomially in k many elementary quantum gates (this is a standard result in quantum computation). We end up with the extra register carrying $\ell(p|_{i-1})$.

Now, φ_i depends only on the location $\ell(p|_{i-1})$ and on the i and $i+1$ qubits. Hence, once again we have a unitary transformation which operates on logarithmically in k many qubits, and so we can implement it in polynomially in k many quantum gates.

After we apply φ_i, we erase the calculation of $\ell(p|_{i-1})$ by applying the inverse of the first transformation which wrote the location down. □

As a corollary, we can deduce that

Corollary 3.5 *For every braid $B \in B_n$, with m crossings, there exists a quantum circuit $Q(B)$ that applies $\varphi(B)$ on n qubits, using* $\mathrm{poly}(m, n, k)$ *elementary gates.*

Proof Order the crossings in the braid in topological order, and apply the corresponding unitary matrix of each crossing, φ_i, one by one, in that order. Each crossing takes $\mathrm{poly}(n, k)$ elementary gates by Claim 3.4, and there are m of them. □

3.3 The Algorithms

We can now describe the algorithms. The input for both is a braid of n strands and m crossings, and an integer k.

Algorithm Approximate-Jones-Plat-Closure

- Repeat for $j = 1$ to $\mathrm{poly}(n, m, k)$:
  1. Generate the state $|\alpha\rangle = |1, 0, 1, 0, \ldots, 1, 0\rangle$
  2. Output a random variable x_j whose expectation value is $\mathcal{R}e\langle\alpha|Q(B)|\alpha\rangle$ using Corollary 3.5 and the Hadamard test.
- Do the same but for random variables y_j whose expectation value is $\mathcal{I}m\langle\alpha|Q(B)|\alpha\rangle$ using the appropriate variant of the Hadamard test.
- Let r be the average over all $x_j + iy_j$ achieved this way. Output $(-A)^{-3w(B^{pl})} \times d^{n/2-1}r$.

Algorithm Approximate-Jones-Trace-Closure

- Repeat for $j = 1$ to $\mathrm{poly}(n, m, k)$:
  1. Classically, pick a random path $p \in P_{n,k}$ with probability $Pr(p) \propto \lambda_\ell$, where ℓ is the index of the site which p ends at.
  2. Output a random variable x_j whose expectation value is $\mathcal{R}e\langle p|Q(B)|p\rangle$ using Corollary 3.5 and the Hadamard test.
- Do the same but for random variables y_j whose expectation value is $\mathcal{I}m\langle p|Q(B)|p\rangle$ using the appropriate variant of the Hadamard test.
- Let r be the average over all $x_j + iy_j$. Output $(-A)^{-3w(B^{\mathrm{tr}})}d^{n-1}r$.

Claim 3.6 *The above two quantum algorithms can be performed in time polynomial in n, m, k.*

Proof Algorithm Approximate-Jones-Plat-Closure is clearly efficient, because the Hadamard test can be applied efficiently using Corollary 3.5. To perform the first step of the second algorithm efficiently, we pick a random $\ell \in \{1, \ldots, k\}$, with probability proportional to $\lambda_\ell \cdot S_{n,\ell}$, where $S_{n,\ell} = |P_{n,k,\ell}|$, namely, the number of paths of n steps on G_k that start at 1 and end at ℓ. We then use Claim 3.7 below to choose one of the paths in $P_{n,k,\ell}$ (almost) uniformly. We note that our calculations involve irrational numbers, λ_ℓ, but these can be approximated to within an exponentially good precision efficiently. □

Claim 3.7 *Given n, k, ℓ, there exists a classical probabilistic algorithm which runs in time polynomial in n and k, and outputs a random path in $P_{n,k,\ell}$ according to a distribution which is exponentially close to uniform.*

Proof (We thank O. Regev for a discussion that led to this variant.) We briefly sketch the proof. We first note that the following $n \times k$ array S, defined by $S_{i,j} = |P_{i,k,j}|$ (the number of paths on G_k of i steps that start at 1 and end at j) can be easily calculated efficiently using the recursion $|P_{n,k,\ell}| = |P_{n-1,k,\ell-1}| + |P_{n-1,k,\ell+1}|$. To pick a random path ending at ℓ, we pick the values of the sites in p one by one in reverse order. If $p(j) = \ell$, we randomly decide if $p(j-1) = \ell - 1$ or $\ell + 1$ according

to the ratio $S_{j-1,\ell-1} : S_{j-1,\ell+1}$. Given $p(j-1)$ we can continue to pick $p(j-2)$ in a similar manner, and so on. □

Theorem 3.8 *Algorithm* Approximate-Jones-Trace-Closure *approximates the Jones polynomial of* B^{tr} *at* $A^{-4} = e^{2\pi i/k}$, *to within the precision specified in Theorem* 1.1.

Proof We will need the following definition.

Definition 3.9 Define $\mathrm{Tr}_n(W)$ for every W in the image of $\Phi(TL_n(d))$ to be:

$$\mathrm{Tr}_n(W) = \frac{1}{N}\sum_{\ell=1}^{k-1} \lambda_\ell \,\mathrm{Tr}(W|_\ell)$$

where $W|_\ell$ denotes the restriction of W to the subspace $\mathcal{H}_{n,k,\ell}$, and Tr denotes the standard trace on matrices. The renormalization is $N = \sum_\ell \lambda_\ell \dim(\mathcal{H}_{n,k,l})$ where the sum is taken over all ℓ's such that $P_{n,k,\ell}$ is non empty.

This definition makes sense because matrices in the image of φ are block diagonal, with the blocks indexed by ℓ, the last site of the paths:

Claim 3.10 *For any* $T \in TL_n(d)$, $\Phi(T)\mathcal{H}_{n,k,\ell} \subseteq \mathcal{H}_{n,k,\ell}$.

Proof Φ_i cannot change the final point of a path since it only moves 01 to 10 and vice versa. □

Hence, the above trace function simply gives different weights to these blocks (and gives zero weights on strings that aren't paths). We claim that Tr_n is a Markov trace.

Claim 3.11 *The function* $\mathrm{Tr}_n(\cdot)$ *satisfies the three properties in Claim* 2.16.

Proof That $\mathrm{Tr}_n(\Phi(1)) = 1$ follows from the renormalization. The second property follows from Claim 3.10 plus the fact that the standard trace on matrices satisfies this property, so $\mathrm{Tr}_n(\cdot)$ satisfies it on each block separately. To show the Markov property, we have to show that if $X \in TL_{n-1}(d))$ then $\mathrm{Tr}_n(\Phi(X)\Phi(E_{n-1})) = \frac{1}{d}\mathrm{Tr}_n(\Phi(X))$. We note that for any $X \in TL_{n-1}(d)$, $\Phi(X)$ can be written as a linear combination of terms of the form $|p\rangle\langle p'| \otimes I$, with $p, p' \in P_{n-1,k}$ and the identity operates on the last qubit. By linearity, it suffices to prove the Markov property on such matrices. Writing $|p\rangle\langle p'| \otimes I = |p0\rangle\langle p'0| + |p1\rangle\langle p'1|$ we require: $\mathrm{Tr}_n(|p0\rangle\langle p'0|\Phi_{n-1} + |p1\rangle\langle p'1|\Phi_{n-1}) = \frac{1}{d}\mathrm{Tr}_n(|p0\rangle\langle p'0| + |p1\rangle\langle p'1|)$. This can be easily verified using the definition of Φ by checking the two cases $p = p'$ and $p \neq p'$.

We start with the case $p \neq p'$. In this case the right hand side is 0. As for the left hand side, $\langle p'0|\Phi_{n-1}$ has a zero component on $\langle p0|$. To see this, we check the two cases: if p' ends with 0 then $\langle p'0|\Phi_{n-1} = 0$, otherwise p' ends with 1. $\langle p'0|\Phi_{n-1}$ is then a sum of two terms, one equals to $\langle p'0|$ and is therefore different than $\langle p0|$, and the other ends with 01 and is thus also different from $\langle p0|$. The same argument works

to show that $\langle p'1|\Phi_{n-1}$ has a zero component on $\langle p1|$. Hence, the left hand side is also 0.

It is left to check the equality in the case $p = p'$. We require

$$\mathrm{Tr}_n(|p0\rangle\langle p0|\Phi_{n-1} + |p1\rangle\langle p1|\Phi_{n-1}) = \frac{1}{d}\,\mathrm{Tr}_n(|p0\rangle\langle p0| + |p0\rangle\langle p0|).$$

Suppose $\ell(p) = \ell$. Then the right hand side is equal to $\frac{1}{dN}(\lambda_{\ell-1} + \lambda_{\ell+1}) = \frac{\lambda_\ell}{N}$, using the properties of the eigenvector λ, as in Claim 2.22. To see that the left hand side is the same, we again divide to cases. Suppose first that p ends with 0. In this case $\langle p0|\Phi_{n-1} = 0$. As for the other term, $\langle p1|\Phi_{n-1} = \frac{\lambda_\ell}{\lambda_{\ell+1}}\langle p1|$ plus a term orthogonal to $\langle p1|$, using the definition of Φ_{n-1} and the fact that p without its last step ends in $\ell + 1$. The weight in the trace of the left hand side is $\lambda_{\ell+1}/N$, and so the left hand side is equal to λ_ℓ/N too. The argument is similar in the case that p ends with 1. □

By the uniqueness of the Markov trace, Lemma 2.17, we have that $\mathrm{Tr}_n(\varphi(B)) = \mathrm{tr}(\rho_A(B))$. Hence, using Lemma 2.18, we have that for any braid $B \in B_n$

Lemma 3.12

$$V_{B^{\mathrm{tr}}}(A^{-4}) = (-A)^{-3w(B^{\mathrm{tr}})} d^{n-1}\,\mathrm{Tr}_n(\varphi(B)).$$

Due to Lemma 3.12, the correctness of the algorithm follows trivially from the following claim.

Claim 3.13 *With all but exponentially small probability, the output r satisfies $|r - \mathrm{Tr}_n(\varphi(B))| \le \varepsilon$ for ε which is inverse polynomial in n, k, m.*

Proof The Hadamard test indeed implies that the expectation of x_j for a fixed p is exactly $\mathcal{R}e\langle p|\varphi(B)|p\rangle$. The expectation of the variable x_j taken over a random p is thus

$$\frac{\sum_{\ell, p\in P_{n,k,\ell}} \lambda_\ell \mathcal{R}e(\langle p|\varphi(B)|p\rangle)}{\sum_{\ell, p\in P_{n,k,\ell}} \lambda_\ell} = \frac{\sum_\ell \lambda_\ell \mathcal{R}e(\mathrm{Tr}(\varphi(B)|_\ell))}{\sum_\ell \lambda_\ell \dim(H_{n,k,l})} = \mathcal{R}e(\mathrm{Tr}_n(\varphi(B))).$$

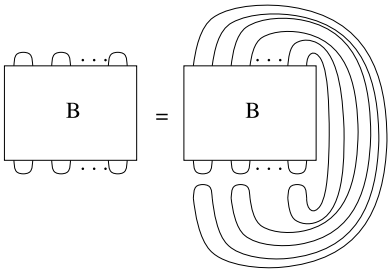
The same argument works for the imaginary part. Since r is the sum of two averages of polynomially many $i.i.d$ random variables, each taking values between 1 and -1, the result follows by the Chernoff-Hoeffding bound. □

This completes the proof of Theorem 3.8. □

Theorem 3.14 *Algorithm* Approximate-Jones-Plat-Closure *approximates the Jones polynomial of B^{pl} at $A^{-4} = e^{2\pi i/k}$, to within the desired precision as in Theorem 1.2.*

Proof By the correctness of the Hadamard test, and by the Chernoff-Hoeffding bound, the variable r which the algorithm computes is, with exponentially good confidence, within $\delta = 1/\mathrm{poly}(n, m, k)$ from $\langle\alpha|\varphi(B)|\alpha\rangle$, which is equal to $\mathrm{Tr}(\varphi(B)|\alpha\rangle \times \langle\alpha|)$. We need to connect this expression to the Jones polynomial of the plat closure

of B. The main observation here is that the plat closure of a braid B is isotopic to the trace closure of a tangle C achieved by applying the braid on $n/2$ capcups, as in the following picture:



It therefore suffices to relate the Jones polynomial of C^{tr} to $\mathrm{Tr}(\varphi(B)|\alpha\rangle\langle\alpha|)$. Since the question is now stated in terms of trace closures and traces, there is hope to be able to apply Lemma 2.18 and Claim 3.11 as in the proof of Theorem 3.8. But we first need to make the connection between the projection on $|\alpha\rangle$ and capcups.

Claim 3.15 $|\alpha\rangle\langle\alpha| = \Phi_1\Phi_3\cdots\Phi_{n-1}/d^{n/2}$.

Proof It is easy to verify that $\Phi_1\Phi_3\cdots\Phi_{n-1}$ applied to any path except for $|\alpha\rangle$ gives 0, and when applied to $|\alpha\rangle$ it gives the desired factor. To do this we use the fact that Φ_i commute if their indices are more than one apart, and so we can first apply Φ_1, then Φ_3 and so on. Since the path starts at the left most site, Φ_1 on p simply applies the following rescaled projection: $d|10\rangle\langle 10|$ on the first two coordinates. This projection forces the first two coordinates to be 10, and so $\Phi_1|p\rangle$ returns to the starting point after two steps. Therefore a similar argument applies when we apply Φ_3 on the next two coordinates, and so on. By induction, we get the desired result. □

We thus have, using Definition 3.9 and Claim 3.15:

$$\begin{aligned}\langle\alpha|\varphi(B)|\alpha\rangle &= \mathrm{Tr}(\varphi(B)|\alpha\rangle\langle\alpha|) = \frac{N}{\lambda_1}\mathrm{Tr}_n(\varphi(B)|\alpha\rangle\langle\alpha|)\\ &= \frac{N}{\lambda_1}\mathrm{Tr}_n(\varphi(B)\Phi_1\Phi_3\cdots\Phi_{n-1}/d^{n/2}) = \frac{N}{\lambda_1 d^{n/2}}\mathrm{Tr}_n(\varphi(C)).\end{aligned}$$

By the uniqueness of the Markov trace (Lemma 2.17), and by Claim 3.11, we have that $\mathrm{Tr}_n(\varphi(C)) = \mathrm{tr}(\rho_A(C))$. We thus have:

$$\langle\alpha|\varphi(B)|\alpha\rangle = \frac{N}{\lambda_1 d^{n/2}}\mathrm{tr}(\rho_A(C)). \tag{5}$$

If we let B in the above equation be the identity braid of n strands, we get that $\frac{N}{\lambda_1} = d^n$ because we know that $\mathrm{tr}(\rho_A(C))$ in this case is equal to $d^{-n/2}$. Hence, we get that for all B,

$$\langle\alpha|\varphi(B)|\alpha\rangle = d^{n/2}\,\mathrm{tr}(\rho_A(C)).$$

Using Lemma 2.18, we have:

$$V_{B^{pl}}(A^{-4}) = V_{C^{\mathrm{tr}}}(A^{-4}) = (-A)^{-3w(C^{\mathrm{tr}})} d^{n/2-1} \langle \alpha | \varphi(B) | \alpha \rangle.$$

This completes the proof of Theorem 3.14. □

Acknowledgements We thank Alesha Kitaev for clarifications regarding the difference between the plat and trace closure, and the nature of the path model representation. We are grateful to Umesh Vazirani for helpful remarks regarding the presentation. We thank Itai Arad, Oded Regev and the anonymous referees for helpful remarks and corrections regarding the technical parts of the paper. The authors are grateful for the ISF, BSF, and ARO grants supporting this research.

References

1. Aharonov, D., Arad, I.: On the BQP-hardness of approximating the Jones polynomial. Preprint (2006)
2. Aharonov, D., Arad, I., Eban, E., Landau, Z.: Polynomial Quantum algorithms for additive approximations of the Potts model and the Tutte polynomial. quant-ph/0702008 (2007)
3. Aharonov, D., Jones, V., Landau, Z.: Preliminary version of this paper. In: Proceedings of STOC (2006). quant-ph/0511096
4. Artin, E.: Theory of braids. Ann. Math. **48**, 101–126 (1947)
5. Bisch, D., Jones, V.: Algebras associated to intermediate subfactors. Invent. Math. **128**, 89–157 (1997)
6. Bordewich, M., Freedman, M., Lovasz, L., Welsh, D.: Approximate counting and quantum computation. Comb. Probab. Comput. **14**(5–6), 737–754 (2005)
7. Childs, A., Cleve, R., Deotto, E., Farhi, E., Gutmann, S., Spielman, D.: Exponential algorithmic speedup by quantum walk. In: STOC (2003)
8. Conway, J.H.: An enumeration of knots and links, and some of their algebraic properties. In: Proceeding of the Conference on Computational Problems in Abstract Algebra, Oxford, 1967, pp. 329–358 (1970)
9. van Dam, W., Hallgren, S.: Efficient quantum algorithms for shifted quadratic character problems. quant-ph/0011067
10. Freedman, M.: P/NP and the quantum field computer. Proc. Natl. Acad. Sci., USA **95**, 98–101 (1998)
11. Freedman, M., Kitaev, A., Larsen, M., Wang, Z.: Topological quantum computation. Mathematical challenges of the 21st century. Bull. Am. Math. Soc. (N.S.) **40**(1), 31–38 (2003)
12. Freedman, M.H., Kitaev, A., Wang, Z.: Simulation of topological field theories by quantum computers. Commun. Math. Phys. **227**, 587–603 (2002)
13. Freedman, M.H., Larsen, M., Wang, Z.: A modular functor which is universal for quantum computation. Commun. Math. Phys. **227**(3), 605–622 (2002)
14. Garnerone, S., Marzuoli, A., Rasetti, M.: quant-ph/0601169
15. Garnerone, S., Marzuoli, A., Rasetti, M.: quant-ph/0606167
16. Garnerone, S., Marzuoli, A., Rasetti, M.: quant-ph/0607203
17. Garnerone, S., Marzuoli, A., Rasetti, M.: quant-ph/0703037v1
18. Goodman, F.M., de la Harpe, P., Jones, V.F.R.: Coxeter Graphs and Towers of Algebras. Springer, Berlin (1989)
19. Hallgren, S.: Polynomial-time quantum algorithms for Pell's Equation and the principal ideal problem. In: STOC, pp. 653–658 (2002)
20. Jaeger, F., Vertigan, D.L., Welsh, D.J.A.: On the computational complexity of the Jones and Tutte polynomials. Math. Proc. Camb. Philos. Soc. **108**(1), 35–53 (1990)
21. Jerrum, M., Sinclair, A., Vigoda, E.: A polynomial-time approximation algorithm for the permanent of a matrix with non-negative entries. In: STOC, pp. 712–721 (2001)
22. Jones, V.F.R.: A polynomial invariant for knots via von Neumann algebras. Bull. Am. Math. Soc. **12**(1), 103–111 (1985)
23. Jones, V.F.R.: Index for subfactors. Invent. Math. **72**, 1–25 (1983)
24. Jones, V.F.R.: Braid groups, Hecke algebras and type II factors. In: Geometric Methods in Operator Algebras. Pitman Research Notes in Math., vol. 123, pp. 242–273. Longman, Harlow (1986)

25. Kauffman, L.: State models and the Jones polynomial. Topology **26**, 395–407 (1987)
26. Kauffman, L.H., Lomonaco, S.J. Jr.: q-deformed spin networks, knot polynomials and anyonic topological computation. J. Knot Theory Ramif. **16**(3), 267–332 (2007). quant-ph/0606114 v3
27. Kauffman, L.H.: Quantum computing and the Jones polynomial. AMS Contemp. Math. Ser. **305**, 101–137 (2002)
28. Kauffman, L.H., Lomonaco, S.J.: A three-stranded quantum algorithm for the Jones polynomial. In: Donkor, E.J., Pirich, A.R., Brandt, H.E. (eds.) Quantum Information and Quantum Computation V. Proceedings of Spie, April 2007, pp. 1–16. Int. Soc. Opt. Eng. arXiv:0706.0020
29. Kuperberg, G.: A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. arXiv:quant-ph/0302112
30. Lomonaco, S.J., Kauffman, L.H.: Topological quantum computing and the Jones polynomial. arXiv:quant-ph/0605004v1
31. Neilsen, M., Chuang, I.: Quantum Computation and Quantum Information. Cambridge Univ. Press, Cambridge (2000)
32. Podtelezhnikov, A., Cozzarelli, N., Vologodskii, A.: Equilibrium distributions of topological states in circular DNA: interplay of supercoiling and knotting. Proc. Natl. Acad. Sci. USA **96**(23), 12974–12979 (1999)
33. Preskill, J.: Topological quantum computation. In: Lecture Notes for Caltech Course # 219 in Physics. http://www.theory.caltech.edu/~preskill/ph229/#lecture
34. Subramaniam, V., Ramadevi, P.: Quantum computation of Jones' polynomials. quant-ph/0210095
35. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. **26**(5), 1484–1509 (1997)
36. Shor, P.W., Jordan, S.P.: Estimating Jones polynomials is a complete problem for one clean qubit. arXiv:quant-ph0707.2831v1
37. Vogel, P.: Representation of links by braids: a new algorithm. Comment. Math. Helv. **65**(1), 104–113 (1990)
38. Watrous, J.: Quantum algorithms for solvable groups. In: STOC, pp. 60–67 (2001)
39. Witten, E.: Quantum field theory and the Jones polynomial. Commun. Math. Phys. **121**(3), 351–399 (1989)
40. Wocjan, P., Yard, J.: The Jones polynomial: quantum algorithms and applications in quantum complexity theory. quant-ph/0603069
41. Wu, F.Y.: Knot theory and statistical mechanics, Rev. Mod. Phys. **64**(4) (1992)